

M7 — Introduction to Modern Data Engineering: student notes

Day 2 (Tue Oct 6), 09:15–09:45. Slides come from Course 2 Module 1 (lake → warehouse → lakehouse); these notes are your take-home version of the discussion.

Why this module matters to you

Lake, data warehouse, lakehouse are not textbook terms in this room — **they are the vocabulary of your own architecture doc.** Your target design (SQL Server DW → BigQuery, Bronze/Silver/Gold) *is* a lakehouse design; this module gives you the industry words for what you're already building, so you can defend the design doc to anyone who asks "why not just a warehouse?" or "why not just a lake?"

Key takeaways

- **The progression:** data warehouses brought structure and performance for BI but are expensive and rigid (schema-on-write, everything modeled before it lands). Data lakes brought cheap, limitless storage for anything (schema-on-read) but decay into "data swamps" without governance. The lakehouse keeps the lake's cheap open storage and bolts the warehouse's reliability and performance on top of it.
- **Why the industry converged on the lakehouse:** running a lake *and* a warehouse side by side means constant data duplication, freshness lag, and engineering overhead shuttling copies between them. The lakehouse collapses those silos into one platform that serves BI, data science, and AI workloads off the same governed data.
- **How it works:** low-cost object storage holds the raw data; a metadata/table layer over it delivers warehouse behavior — ACID transactions, schema enforcement and evolution, indexing, access control.
- **The key lakehouse features:** low-cost, scalable object storage; open file formats (Parquet) and open table formats (Apache Iceberg) so no single engine owns your data; ACID transactions and schema evolution on files; time travel and rollback; storage and compute that scale independently; multiple engines (BigQuery, Spark, others) working safely on the same tables; unified, fine-grained governance across all of it.
- **'Open' is the load-bearing word.** Formats and table layers are industry standards — your Bronze files stay readable by non-Google engines, which is your insurance against lock-in.
- **In your terms:** the lake = Cloud Storage (your Bronze); the warehouse = BigQuery (your Silver/Gold); the lakehouse = the two acting as one system, with BigQuery as the processing engine over both.
- The deck's built-in quiz at the end runs as a group — it's a vocabulary check, not a grade. If you can answer it, you can narrate your own architecture doc.

Your SQL Server world → GCP world

Your SQL Server world	GCP world (this module's terms)
SQL Server data warehouse (the single source of truth)	BigQuery — the warehouse half of the lakehouse
Network shares and file-drop folders where extracts pile up	Cloud Storage buckets — the data lake / your Bronze layer
"Everything must be modeled before it lands"	Schema-on-read: land raw in Bronze first, model into Silver/Gold when ready
Hand-rolled validation and staging databases keeping loads honest	Lakehouse table features: ACID transactions, schema enforcement and evolution
SSAS cubes and rigid, pre-modeled marts	Open table formats (Iceberg) plus BigQuery-managed Gold marts — flexible but governed
One engine (the SQL Server instance) owns the data	Storage and compute decoupled — GCS owns the bytes, BigQuery (or Spark) borrows them

Dig deeper

- [What is a data lakehouse? \(Google Cloud\)](#) — includes a lake vs. warehouse vs. lakehouse comparison table worth reusing in your own docs
- [Apache Iceberg](#) — the open table format behind the features above
- [Introduction to BigLake tables](#) — how BigQuery puts governance on data lake files

Part of the Iowa DOM DoIT Data Engineering student pack — see ../00_schedule.md.